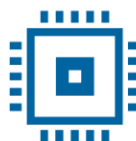




Universitatea
Transilvania
din Brașov



Universitatea
Transilvania
din Brașov

FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

Proiect de semestru la disciplina Roboți Mobili

Autor: Crivilescu Adrian-Dragoș

Programul de studii: Robotică

Grupa: 4LF821

Coordonator: Prof. Dr. Ing. Pozna Claudiu

Cuprins

1	Temă de proiect	3
2	Componentele robotului mobil	4
3	Provocările proiectului	5
	3.1 Senzori și encodere	5
	3.2 Mecanice și de structură	5
	3.3 Alimentare și putere	6
	3.4 Control și stabilitate	6
4	Traseul primitiv: Trapezul	7
	4.1 Implementarea în Arduino IDE	7
	4.2 Modelele matematice utilizate	7
	4.3 Algoritmul de control	8
	4.4 Execuție practică a traseului	8
5	Trasee complexe	10
	5.1 Interfața grafică în Python	10
	5.2 Implementarea în Arduino IDE	11
	5.3 Modele matematice utilizate	12
	5.4 Algoritmul de control	13
	5.5 Execuția practică a traseului	14
6	Concluzii	15

Capitolul 1

Temă de proiect

Proiectul constă în realizarea unui robot mobil autonom care execută traiectorii prestabilite prin controlul precis al motoarelor DC, folosind un microcontroler Arduino Uno și senzori de rotație (encodere optice). Controlul vitezei și al direcției se face prin algoritmi PID implementați în Arduino IDE.

Pentru traseele primitive, în special traseul trapezului, codul gestionează direct execuția în Arduino, realizând rotații și deplasări liniare controlate.

Pentru trasee complexe, utilizatorul definește traiectoria printr-o interfață grafică dezvoltat în Python, care generează coordonate ce sunt încărcate robotului.

Robotul urmărește traiectoria cu precizie, ajustând continuu viteza motoarelor pentru corecția diferențelor de rotație detectate prin encodere, asigurând stabilitatea și acuratețea deplasării.

Capitolul 2

Componentele robotului mobil

Platforma hardware:

- Kit preconstruit de robot mobil cu 4 motoare DC și control diferențial stânga/dreapta.

Controler și drivere:

- Microcontroler Arduino Uno.
- Driver motoare L298N pentru controlul motoarelor DC.

Motoare și senzori:

- 4 motoare pe curent continuu cu roți atașate.
- Encodere optice integrate pentru măsurarea rotației roților.

Alimentare:

- Kit de 8 baterii AA pentru alimentarea sistemului.

Funcționalitate specifică:

- Motoarele permit deplasare înainte și înapoi.
- Viteza motoarelor este controlată prin semnale PWM generate de Arduino.
- Tracțiune diferențială — fiecare pereche de roți (stânga și dreapta) se controlează independent.
- Semnalele PWM trebuie să depășească pragul de aproximativ 20% din maxim pentru a porni motoarele.

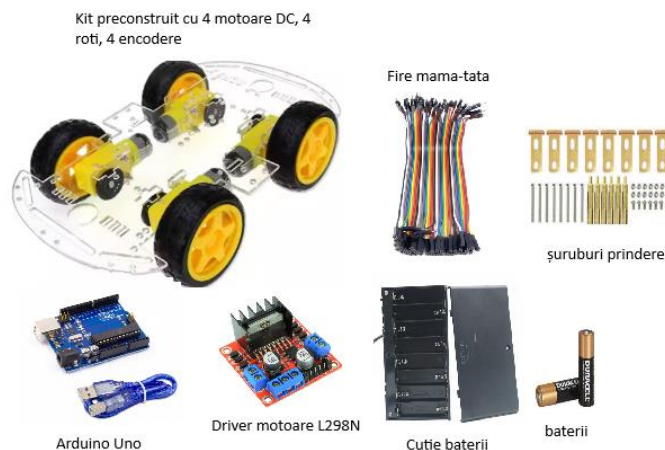


Figura 1 – Componentele robotului mobil montat

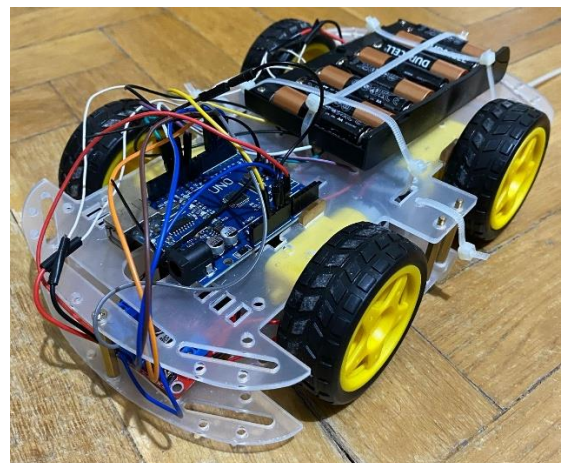


Figura 2 – Robotul

Capitolul 3

Provocările proiectului

În procesul realizării proiectului, s-au identificat o serie de provocări tehnice ce au influențat implementarea și funcționarea robotului mobil.

3.1 Senzori și encodere

Probleme:

- Kitul conținea diode emițător-receptor nepotrivite pentru citirea impulsurilor din encoder.
- Citirea impulsurilor nu a fost constantă din cauza poziționării neoptime.
- Conexiunile electrice cu conectori Dupont au fost instabile, generând întreruperi ale semnalului.
- Accesul dificil la senzori după asamblare a limitat intervențiile în cazul defecțiunilor (un encoder s-a ars în teste).

Soluții:

- Senzorii pentru encodere au fost cumpărați separat, iar montarea lor în șasiu a fost dificilă, necesitând fixare cu superglue pentru poziționare optimă
- Fixarea cu superglue a senzorilor pentru poziționare stabilă.
- Ar fi fost posibilă utilizarea mediei semnalelor de la mai mulți encodere pentru reducerea erorilor, dar nu a fost implementată din cauza problemelor hardware.

3.2 Mecanice și de structură

Probleme:

- Lipsa unor piulițe și șuruburi
- Axele motoarelor necentrate generau vibrații puternice în funcționare, care provocau desprinderi ale roților de encodere
- Cauciucul roților era de calitate slabă: alunecă pe suprafețe netede și se prinde excesiv pe suprafețe rugoase.

Soluții:

- rigidizarea șasiului cu coliere de plastic pentru stabilitate.
- Lipirea roților de encodere pe axele motoarelor cu superglue pentru stabilitate.

- Posibilă soluție viitoare: creșterea greutateii robotului pentru îmbunătățirea aderenței.

3.3 Alimentare și putere

Problemă:

- Setul inițial de 4 baterii AA se consuma rapid și nu asigura tensiune suficientă pentru motoare, chiar și la 100% PWM.

Soluție:

- Adoptarea unei cutii de 8 baterii AA pentru tensiune mai stabilă și suficientă, asigurând funcționarea corectă a motoarelor.

3.4 Control și stabilitate

Problemă:

- Probleme hardware precum vibrațiile și conexiunile instabile afectau controlul precis al traiectoriei.

Soluții:

- Implementarea unui control PID pentru corectarea diferențelor de viteză între roțile stânga și dreapta.
- Folosirea unui factor de corecție experimental pentru calibrarea odometriei.

Capitolul 4

Traseul primitiv: Trapezul

4.1 Implementarea în Arduino IDE

Traseul descris de robot este un trapez dreptunghic, în care robotul urmează traseul descris în figura 3.

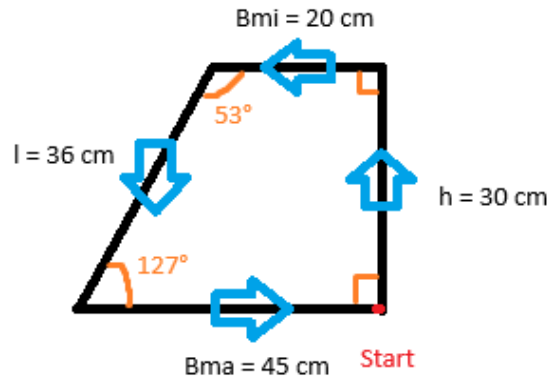


Figura 3 – Traseul primitiv

Codul a fost scris în Arduino IDE și se bazează pe controlul prin impulsuri de la doi senzori de encoder, câte unul pe fiecare parte.

4.2 Modelele matematice utilizate

Pentru ca robotul să poată urma cu precizie un traseu predefinit de forma unui trapez, a fost nevoie de transpunerea traiectoriei geometrice în **mișcări de rotație și translație**, controlate prin encoder.

Mișcarea este modelată în două etape:

1. Rotirea robotului în direcția dorită

Unghiul pe care trebuie să îl parcurgă robotul este exprimat în radiani și se transformă în impulsuri necesare pentru a roti una dintre roți (cealaltă fiind oprită). Formula utilizată este:

$$N_0 = \frac{L \cdot \theta}{R} \cdot F_r \quad (4.0)$$

unde:

N_0 = numărul de impulsuri pentru rotație,

L = distanța dintre roțile stânga și dreapta (ecartamentul),

θ = unghiul de rotit (în radiani),

R = raza roții (jumătate din diametrul acesteia),

F_r = factor de corecție pentru rotație (empiric, în funcție de frecarea roților și suprafață).

2. Deplasarea în linie dreaptă către următorul punct.

Robotul execută un traseu fix, unde distanțele de parcurs sunt cunoscute. Prin urmare, nu este nevoie de calculul geometric al distanței între puncte, ci doar de **translatarea unei distanțe liniare în tick-uri de encoder**.

$$N_d = d \cdot T_{cm} \cdot F_l \quad (4.1)$$

unde:

N_d = numărul de tick-uri encoder necesare pentru a parcurge distanța d (în cm),

T_{cm} = numărul de tick-uri pe cm (determinat în funcție de roată și encoder),

F_l = factor de corecție pentru deplasare în linie dreaptă (determinare empirică).

Valoarea lui T_{cm} este calculată cu relația:

$$T_{cm} = \frac{T_{rot}}{C} \quad (4.2)$$

unde:

T_{rot} = tick-uri per rotație completă (în cazul nostru, 20),

C = circumferința roții ($C = \pi \cdot D$) cu $D=6.5$ cm

4.3 Algoritmul de control

Controlul robotului este implementat în pași simpli și determinist, fără bucle PID avansate în această versiune:

1. **Resetarea impulsurilor** înainte de fiecare mișcare (`resetPulses()`).
2. **Pornirea motoarelor** în direcția și cu viteza dorite (`setMotoare()`).
3. **Așteptarea atingerii numărului dorit de impulsuri** folosind `getLeftAverage()` și `getRightAverage()` în bucle while.
4. **Oprirea motoarelor** imediat după atingerea distanței dorite (`opresteMotoarele()`).

Pentru rotații, doar un motor este acționat, ceea ce simplifică controlul dar poate introduce o ușoară eroare de traiectorie.

4.4 Execuția practică a traseului

Pentru a valida corectitudinea traseului trapezoidal generat prin codul Arduino, au fost realizate mai multe capturi din timpul deplasării robotului. Acestea surprind pozițiile intermediare cheie și confirmă urmărirea fidelă a traseului programat.



Figura 4 – Pozitie inițială (inceput h)



Figura 5 – Poziție intermediară (capat h)



Figura 6 – Pozitie intermediară
(sfârșit bază mică)



Figura 7 – Poziție intermediară
(capăt latură diag.)



Figura 8 – Pozitie intermediară (rotație)



Figura 9 – Poziție finală

Capitolul 5

Trasee complexe

În continuarea experimentelor cu trasee prestabilite, acest capitol urmărește extinderea capabilităților robotului mobil prin introducerea unei metode mai flexibile de generare a traiectoriilor. Se propune o abordare în care utilizatorul poate desena liber o traiectorie folosind o interfață grafică (GUI) realizată în Python. Această interfață permite obținerea unei secvențe de puncte (x, y) care descriu traseul dorit, puncte ce pot fi exportate automat într-un fișier și apoi transmise către Arduino pentru execuție.

5.1 Interfața grafică în Python

Pentru definirea traiectoriei robotului, a fost realizată o interfață grafică în Python folosind Tkinter.

Utilizatorul poate desena traseul cu mouse-ul, iar coordonatele (x, y) sunt colectate și salvate într-un fișier.

Interfața include opțiuni precum:

- exportul datelor în format pentru Arduino și CSV în funcție de variabile
- ștergerea desenului
- previzualizarea punctelor cu funcție de zoom in-out și navigare
- sub-eșantionarea: de exemplu, se setează pe 10, va salva un punct din 10
- pixel -> cm: dimensiunea unui pixel în cm

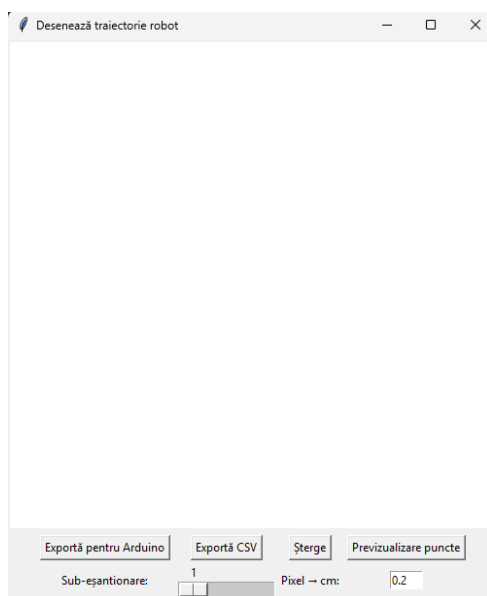


Figura 10 – Interfața grafică pentru definire traiectorie

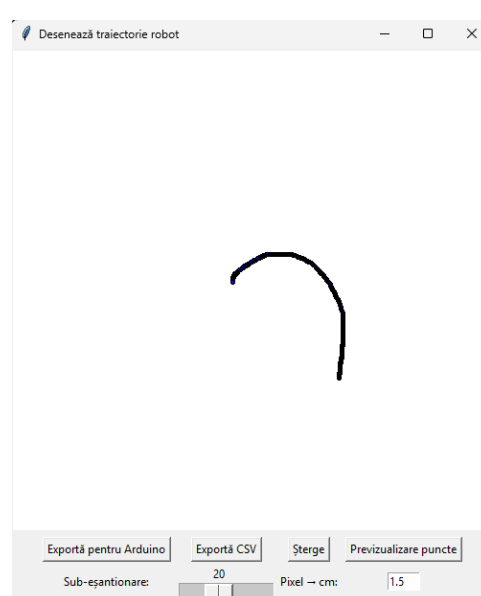


Figura 11 – Traiectorie trasată în GUI

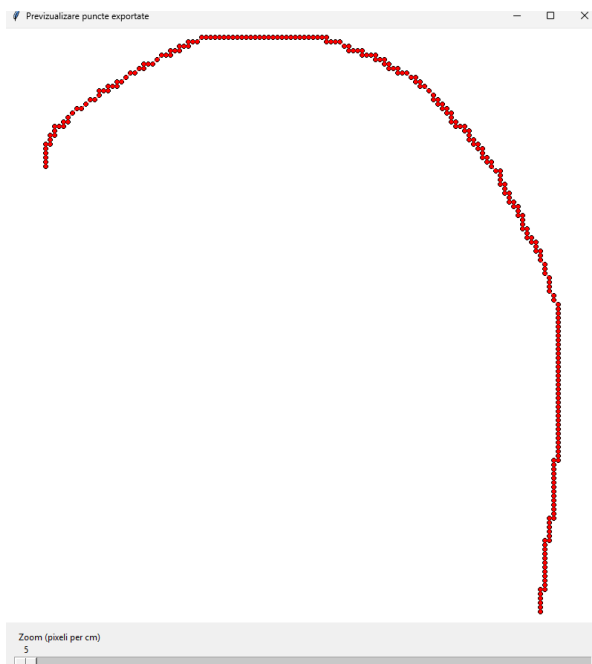


Figura 12 – Previzualizarea punctelor cu sub-eșantionare 1

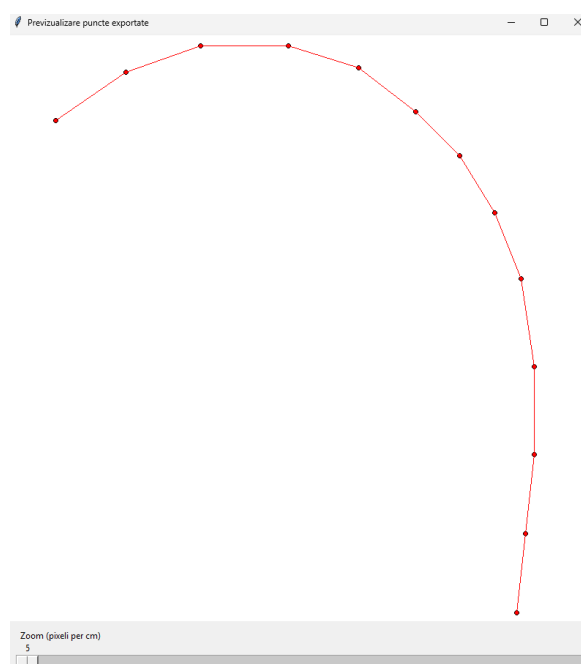


Figura 13 – Previzualizarea punctelor cu sub-eșantionare 20

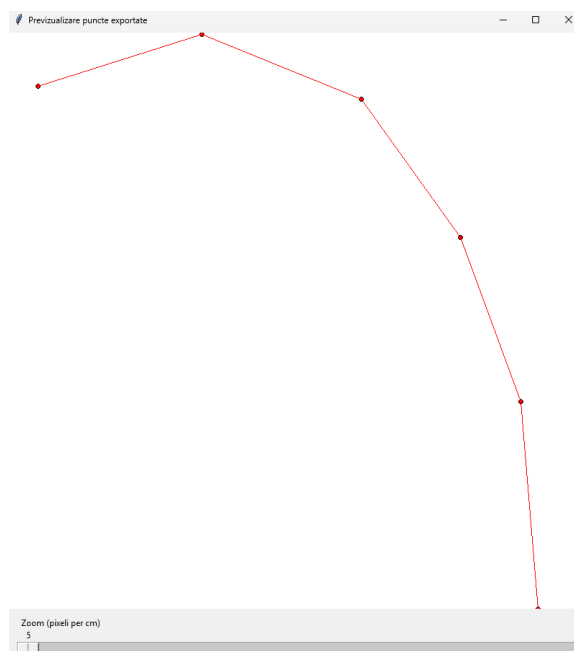


Figura 14 – Previzualizarea punctelor cu sub-eșantionare 50

5.2 Implementarea în Arduino IDE

Fișierul `traietorie_arduino.h` generat din GUI-ul Python conține punctele traiectoriei. În Arduino IDE, fiecare punct este parcurs secvențial.

Robotul:

- Calculează unghiul către punctul următor și execută rotația;
- Apoi se deplasează în linie dreaptă pe baza impulsurilor encoderelor;
- La fiecare pas, poziția este actualizată folosind o formulă simplă de odometrie;
- Controlul este în buclă deschisă, cu o corecție proporțională (P) pentru menținerea direcției.

Astfel, robotul urmează traiectoria fără feedback din senzori externi, doar cu ajutorul encoderelor

5.3 Modele matematice utilizate

Pentru a executa o traiectorie formată din puncte (x, y), robotul mobil trebuie să efectueze o succesiune de rotații și deplasări. La fiecare pas, se calculează:

1. Unghiul de orientare către următorul punct:

$$\Delta\theta = \text{atan2}(y_{urm} - y_{curent}, x_{urm} - x_{curent}) - \theta_{curent} \quad (5.0)$$

2. Distanța până la următorul punct

$$d = \sqrt{(x_{urm} - x_{curent})^2 + (y_{urm} - y_{curent})^2} \quad (5.1)$$

3. Calculul tick-urilor encoder pentru deplasare:

- Pentru rotire:

$$t_r = \frac{|\Delta\theta| \cdot L \cdot \pi}{360 \cdot R} \cdot TPR \cdot f \quad (5.2)$$

- Pentru mers înainte:

$$t_d = d \cdot TPC \cdot f \quad (5.3)$$

unde:

L – ecartamentul robotului (distanța dintre roțile din stânga și din dreapta)

R – raza roții

TPR – tick-uri pe rotație completă a roții

TPC = TPR / (2*π*R) – tick-uri pe centimetru

f – factor de corecție empiric (experimentat pe robotul real)

Pentru a corecta diferențele între cele două roți (ex: alunecare, nealiniere), s-a folosit o strategie simplă bazată pe un **controler proporțional (P)**. Se calculează eroarea dintre numărul de tick-uri de pe partea stângă și dreaptă:

$$\text{error} = \text{ticks}_{\text{left}} - \text{ticks}_{\text{right}} \quad (5.4)$$

$$\text{output} = K_p \cdot \text{error} \quad (5.5)$$

PWM-urile trimise la motoare sunt ajustate astfel:

$$PWM_{\text{left}} = PWM_{\text{nominal}} - \text{output} \quad (5.6)$$

$$PWM_{\text{right}} = PWM_{\text{nominal}} + \text{output} \quad (5.7)$$

Acest mecanism permite robotului să mențină o direcție rectilinie chiar și în prezența unor erorimecanice sau diferențe între motoare. Valoarea lui K_p a fost aleasă empiric (ex: 0.7), iar componenta integrală și derivativă (I și D) sunt dezactivate.

După fiecare mișcare (rotație sau deplasare), poziția robotului este actualizată folosind **odometria**:

$$\delta s = \frac{d_{\text{left}} + d_{\text{right}}}{2} \quad (5.8)$$

$$\delta \theta = \frac{d_{\text{right}} - d_{\text{left}}}{L} \quad (5.9)$$

$$x_{\text{nou}} = x + \delta s \cdot \cos(\theta + \delta \theta / 2) \quad (5.10)$$

$$y_{\text{nou}} = y + \delta s \cdot \sin(\theta + \delta \theta / 2) \quad (5.11)$$

$$\theta_{\text{nou}} = \theta + \delta \theta \quad (5.12)$$

unde:

δs - media dintre distanța parcursă de roata stângă și cea dreaptă.

$\delta \theta$ - modificarea unghiului de orientare, Dacă $d_{\text{Right}} > d_{\text{Left}}$, robotul a virat la dreapta.

x_{Nou} – actualizare poziția pe axa X

y_{Nou} – actualizare poziția pe axa Y

θ_{Nou} – actualizare unghiul de orientare

5.4 Algoritmul de control

Algoritmul de control al robotului mobil are rolul de a executa o traiectorie definită prin puncte (x,y) folosind mișcări secvențiale de rotație și deplasare înainte. Controlul se bazează pe informațiile furnizate de encodere și pe aplicarea unui control proporțional pentru menținerea traiectoriei drepte.

Structura algoritmului este următoarea:

1. **Parcurea punctelor** din traiectorie două câte două ($P_i \rightarrow P_{i+1}$)
2. **Calculul unghiului** dintre punctele curente și determinarea unghiului de rotație necesar.
3. **Rotirea robotului** cu o funcție de tip „rotire_la_unghi”, pe baza diferenței de orientare.
4. **Deplasarea înainte** pe distanța calculată între cele două puncte.

5. În timpul mersului, se aplică un **control proporțional (P)**
6. După fiecare acțiune, se actualizează poziția și orientarea curentă folosind odometria.

Acest algoritm asigură un control simplu, dar eficient, pentru execuția traiectoriilor prestabilite, compensând micile diferențe de viteză între roți și oferind o deplasare coerentă.

5.4 Execuția practică a traseului

În această etapă, traiectoria din Figura 14 este executată de robot pe baza comenzilor de deplasare și rotație. Se testează funcționarea sistemului în condiții reale, urmărindu-se respectarea punctelor traseului și corectarea direcției folosind datele de la encodere.

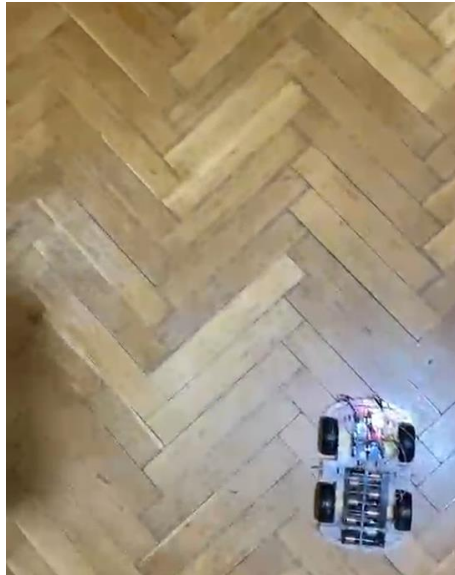


Figura 15 – poziție inițială

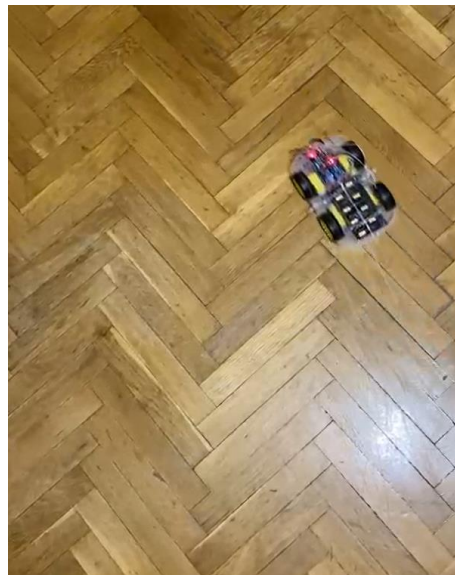


Figura 16 – poziție intermediară

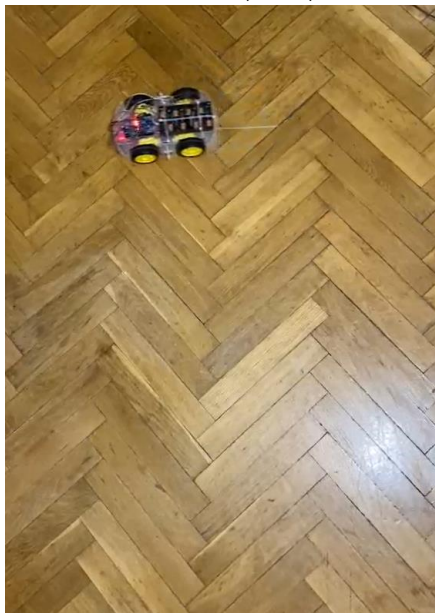


Figura 17 – poziție intermediară

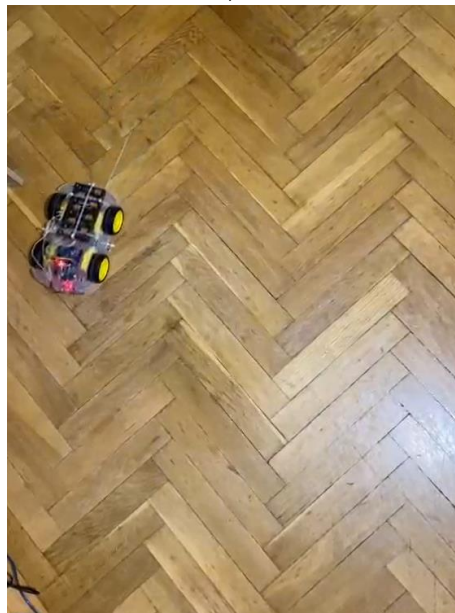


Figura 18 – poziție finală

Capitolul 6

Concluzii

Proiectul realizat a demonstrat fezabilitatea construirii și controlului unui robot mobil capabil să urmeze traiectorii definite pe baza unor puncte preînregistrate. Utilizarea senzorilor de tip encoder a permis măsurarea precisă a distanței parcurse și a unghiurilor de rotație, oferind feedback esențial pentru corectarea traiectoriei în timp real.

Implementarea unui control proporțional (P) a contribuit la menținerea unei deplasări echilibrate între cele două roți, reducând abaterile datorate diferențelor mecanice sau alunecării. Odometria a fost folosită pentru estimarea poziției curente a robotului, cu rezultate bune în condiții experimentale.

Execuția practică a traseului a confirmat funcționalitatea algoritmilor dezvoltați, cu o precizie satisfăcătoare pentru aplicații educaționale sau demonstrative. Deși pot apărea mici erori de poziționare din cauza derivelor acumulate în timp, soluția propusă oferă o bază solidă pentru dezvoltări ulterioare, cum ar fi integrarea unor senzori suplimentari sau trecerea la un control PID complet.